# Alumni Influencers

## *Database Schema Documentation*

Version 1.0.0
Developer: Phantasmagoria Ltd
Database: MongoDB (NoSQL) with Mongoose ODM
*Relational Model: Reference-based relationships via ObjectId*

1. Overview ..... 3
1.1 Design Principles ..... 3
1.2 Technology Stack ..... 3
2. Entity Relationships ..... 4
2.1 Relationship Summary ..... 5
3. Entity: User ..... 5
3.1 Schema Fields ..... 5
3.2 Enum Values ..... 6
3.3 Embedded Sub-Document: loginHistory ..... 6
3.4 Indexes ..... 6
3.5 Mongoose Schema (Reference) ..... 6
4. Entity: Profile ..... 7
4.1 Schema Fields ..... 7
4.2 Embedded Sub-Document: degrees[ ] ..... 8
4.3 Embedded Sub-Document: certifications[ ] ..... 8
4.4 Embedded Sub-Document: employmentHistory[ ] ..... 8
4.5 Indexes ..... 8
4.6 Mongoose Schema (Reference) ..... 9
5. Entity: Bid ..... 10
5.1 Schema Fields ..... 10
5.2 Enum Values ..... 10
5.3 Bid Lifecycle ..... 11

5.4 Indexes ..... 11
5.5 Mongoose Schema (Reference) ..... 11
6. Entity: API Key ..... 12
6.1 Schema Fields ..... 12
6.2 Embedded Sub-Document: usageStats[ ] ..... 12
6.3 Indexes ..... 12
6.4 Mongoose Schema (Reference) ..... 13
7. Collection Quick Reference ..... 14

# 1. Overview

The Alumni Influencers platform uses MongoDB as its primary data store, managed through the Mongoose ODM (Object Document Mapper). Although MongoDB is a document-oriented NoSQL database, the schema is designed with well-defined entity relationships enforced at the application layer via Mongoose schemas and validation rules.

The database consists of four core collections:

- users — core authentication and account management
- profiles — extended alumni information displayed to the public
- bids — the auction engine for the Alumni of the Day feature
- apikeys — security credentials issued to third-party client applications

## 1.1 Design Principles

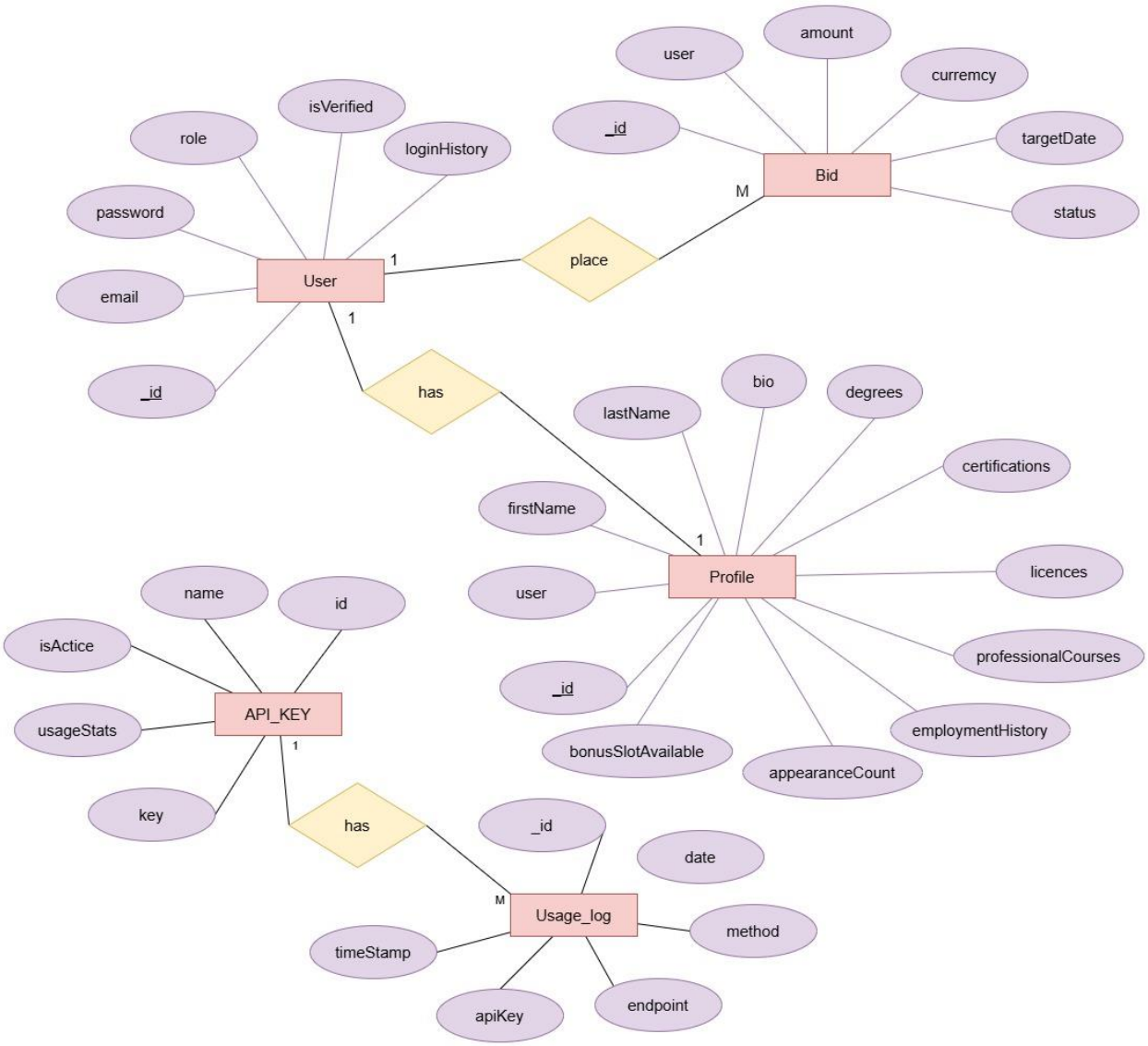
- References over embedding for large or independently queried documents (e.g. Profile references User rather than embedding user data)
- Embedding for small, tightly coupled sub-documents that are always accessed together (e.g. degrees, certifications within a Profile)
- Unique indexes enforced on natural keys such as email and API key strings
- All timestamps stored in UTC ISO-8601 format

## 1.2 Technology Stack

| Component | Detail |
|---|---|
| Database | MongoDB (NoSQL Document Store) |
| ODM | Mongoose v7+ |
| ID Type | ObjectId (BSON) — auto-generated by MongoDB |
| Timestamps | UTC ISO-8601 (e.g. 2026-02-28T10:00:00.000Z) |
| Password Hashing | bcrypt (salted, 10 rounds) |
| Currency | GBP (default) / LKR |

# 2. Entity Relationships

The diagram below describes the logical relationships between the four primary entities. Relationships are enforced via ObjectId references at the application layer.



## 2.1 Relationship Summary

**USER - PROFILE**
*One-to-one: every user has exactly one profile. Profile references User via ObjectId.*

**USER - BID**
*One-to-many: a user can place many bids across different target dates. Each bid references its owner User via ObjectId.*

**API_KEY - USAGE_LOG**
*One-to-many: each API key accumulates an array of usage log entries (endpoint, method, timestamp).*

# 3. Entity: User

Collection: users
The core authentication entity. Every person who interacts with the platform has a User document. Role-based access control is managed through the role field, while email verification is tracked via isVerified.

## 3.1 Schema Fields

| Field | Type | Constraints | Description |
|---|---|---|---|
| _id | ObjectId | PK, auto | Unique document identifier, auto-generated by MongoDB. |
| email | String | Unique, Required | University email address (.ac.uk or .edu domains only). |
| password | String | Required | bcrypt-hashed password. Never stored in plaintext. |
| role | String | Enum, Default: user | Access level. Either 'user' (alumni) or 'admin'. |
| isVerified | Boolean | Default: false | Set to true upon successful email verification click. |
| loginHistory | Array | Embedded | Log of past login events (timestamp, IP, user agent). |
| createdAt | Date | Auto (Mongoose) | Document creation timestamp (UTC). |
| updatedAt | Date | Auto (Mongoose) | Last modification timestamp (UTC). |

## 3.2 Enum Values

| Field | Allowed Values |
|---|---|
| role | **user** \| **admin** |

## 3.3 Embedded Sub-Document: loginHistory

Each entry in the loginHistory array captures contextual data about a successful authentication event.

| loginHistory[ ] Field | Type | Description |
|---|---|---|
| timestamp | Date | UTC time of the login event. |
| ip | String | IPv4 or IPv6 address of the client. |
| userAgent | String | Browser / device user-agent string. |

## 3.4 Indexes

| Field | Type | Constraints | Description |
|---|---|---|---|
| email | String | Unique Index | Enforces email uniqueness across all user accounts. |

## 3.5 Mongoose Schema (Reference)

```
const UserSchema = new mongoose.Schema({
  email:       { type: String, required: true, unique: true, lowercase: true },
  password:    { type: String, required: true },
  role:        { type: String, enum: ['user', 'admin'], default: 'user' },
  isVerified:  { type: Boolean, default: false },
  loginHistory: [{ timestamp: Date, ip: String, userAgent: String }]
}, { timestamps: true });
```

# 4. Entity: Profile

Collection: profiles
Extended alumni data linked one-to-one with a User document. This is the primary public-facing entity.its contents are displayed on the AR client when an alumnus wins the daily bid. All academic credentials and professional history are embedded as arrays of sub-documents within the Profile.

## 4.1 Schema Fields

| Field | Type | Constraints | Description |
|---|---|---|---|
| _id | ObjectId | PK, auto | Unique document identifier. |
| user | ObjectId | FK → users, Unique | Reference to the owning User document. |
| firstName | String | Required | Alumni's first name. |
| lastName | String | Required | Alumni's last name. |
| bio | String | Optional | Short biography displayed on the AR client. |
| profileImage | String | Optional | Stored filename of the uploaded JPEG/PNG image. |
| degrees | Array | Embedded | Academic degree records. See sub-document schema below. |
| certifications | Array | Embedded | Professional certifications (e.g. AWS, PMP). |
| licences | Array | Embedded | Professional licences (e.g. medical, legal). |
| professionalCourses | Array | Embedded | Short courses and CPD records. |
| employmentHistory | Array | Embedded | Chronological work history. |
| appearanceCount | Number | Default: 0 | Total times this profile has been the Alumni of the Day. |
| bonusSlotAvailable | Boolean | Default: false | Flag granting a free additional bid slot. |
| createdAt | Date | Auto (Mongoose) | Document creation timestamp (UTC). |
| updatedAt | Date | Auto (Mongoose) | Last modification timestamp (UTC). |

## 4.2 Embedded Sub-Document: degrees[ ]

| degrees[ ] Field | Type | Description |
|---|---|---|
| degreeTitle | String | Title of the degree (e.g. BSc Computer Science). |
| university | String | Name of the awarding institution. |
| completionDate | Date | Date the degree was awarded (UTC). |
| certificateUrl | String | Optional URL to a digital certificate or transcript. |

## 4.3 Embedded Sub-Document: certifications[ ]

| certifications[ ] Field | Type | Description |
|---|---|---|
| title | String | Name of the certification (e.g. AWS Solutions Architect). |
| issuer | String | Issuing body or organisation. |
| issuedDate | Date | Date the certification was issued. |
| expiryDate | Date | Optional expiry date (null if no expiry). |
| badgeUrl | String | Optional URL to the digital badge. |

## 4.4 Embedded Sub-Document: employmentHistory[ ]

| employmentHistory[ ] Field | Type | Description |
|---|---|---|
| jobTitle | String | Job title / role held. |
| company | String | Name of the employer. |
| startDate | Date | Employment start date (UTC). |
| endDate | Date | Employment end date. Null if currently employed. |
| description | String | Optional summary of responsibilities. |

## 4.5 Indexes

| Field | Type | Constraints | Description |
|---|---|---|---|
| user | ObjectId | Unique Index | Enforces one-to-one relationship — one profile per user. |

## 4.6 Mongoose Schema (Reference)

```
const ProfileSchema = new mongoose.Schema({
  user:          { type: ObjectId, ref: 'User', required: true, unique: true },
  firstName:     { type: String, required: true },
  lastName:      { type: String, required: true },
  bio:           { type: String },
  profileImage:   { type: String },
  degrees:       [{ degreeTitle: String, university: String,
                  completionDate: Date, certificateUrl: String }],
  certifications: [{ title: String, issuer: String,
                  issuedDate: Date, expiryDate: Date, badgeUrl: String }],
  licences:      [{ title: String, issuer: String, issuedDate: Date }],
  professionalCourses: [{ title: String, provider: String, completionDate: Date }],
  employmentHistory:   [{ jobTitle: String, company: String,
                    startDate: Date, endDate: Date, description: String }],
  appearanceCount:     { type: Number, default: 0 },
  bonusSlotAvailable:  { type: Boolean, default: false }
}, { timestamps: true });
```

# 5. Entity: Bid

Collection: bids
The auction management entity. Each document represents a single bid placed by an alumnus for a specific future date. The system operates as a blind auction: users are only informed of their own bid status (Winning or Losing) without visibility into competitor amounts.

## 5.1 Schema Fields

| Field | Type | Constraints | Description |
|---|---|---|---|
| _id | ObjectId | PK, auto | Unique document identifier. |
| user | ObjectId | FK → users, Required | Reference to the User who placed the bid. |
| amount | Number | Required, Min: 0 | The bid value in the specified currency. |
| currency | String | Enum, Default: GBP | Currency of the bid amount. |
| targetDate | Date | Required | The midnight UTC date the user is bidding to be featured. |
| status | String | Enum, Required | Current lifecycle state of the bid. |
| createdAt | Date | Auto (Mongoose) | Timestamp when the bid was first placed. |
| updatedAt | Date | Auto (Mongoose) | Timestamp of the most recent update (e.g. amount increase). |

## 5.2 Enum Values

| Field | Allowed Values |
|---|---|
| status | **active** \| **won** \| **lost** \| **superseded** \| **cancelled** |

| Field | Allowed Values |
|---|---|
| currency | **GBP** \| **LKR** |

## 5.3 Bid Lifecycle

| Status | When Applied |
|---|---|
| active | Bid has been placed and is currently competing for the target date. |
| won | This bid had the highest amount at the daily cut-off time. |
| lost | Another bid was higher at the cut-off time. |
| superseded | The user increased their bid; the old document is marked superseded and a new one created. |
| cancelled | The user or an admin cancelled the bid before the cut-off. |

## 5.4 Indexes

| Field | Type | Constraints | Description |
|---|---|---|---|
| user + targetDate | Compound | Unique Index | Prevents a user from having more than one active bid per target date. |
| targetDate | Date | Index | Optimises daily winner resolution queries. |

## 5.5 Mongoose Schema (Reference)

```
const BidSchema = new mongoose.Schema({
  user:       { type: ObjectId, ref: 'User', required: true },
  amount:     { type: Number, required: true, min: 0 },
  currency:   { type: String, enum: ['GBP', 'LKR'], default: 'GBP' },
  targetDate: { type: Date, required: true },
  status:     { type: String,
              enum: ['active','won','lost','superseded','cancelled'],
              required: true }
}, { timestamps: true });

BidSchema.index({ user: 1, targetDate: 1 }, { unique: true });
BidSchema.index({ targetDate: 1 });
```

# 6. Entity: API Key

Collection: apikeys
Security credentials issued by administrators to authorise third-party client applications (such as the AR kiosk). Each key has an activation state that can be toggled, and an embedded usage log for audit purposes.

## 6.1 Schema Fields

| Field | Type | Constraints | Description |
|---|---|---|---|
| _id | ObjectId | PK, auto | Unique document identifier. |
| name | String | Unique, Required | Human-readable label (e.g. 'AR Client v1'). |
| key | String | Unique, Required | The hashed secure string used in the x-api-key header. |
| isActive | Boolean | Default: true | When false, all requests using this key are rejected. |
| usageStats | Array | Embedded | Append-only log of usage events. See sub-document below. |
| createdAt | Date | Auto (Mongoose) | Timestamp when the key was generated. |
| updatedAt | Date | Auto (Mongoose) | Timestamp of the most recent status change. |

## 6.2 Embedded Sub-Document: usageStats[ ]

| usageStats[ ] Field | Type | Description |
|---|---|---|
| date | Date | UTC timestamp of the API request. |
| endpoint | String | The path that was accessed (e.g. /api/client/alumni-of-the-day). |
| method | String | HTTP method used (e.g. GET). |
| ip | String | Optional client IP address of the requesting application. |

## 6.3 Indexes

| Field | Type | Constraints | Description |
|---|---|---|---|
| name | String | Unique Index | Enforces unique labels across all API keys. |
| key | String | Unique Index | Enforces uniqueness of the secret key string. |
| isActive | Boolean | Index | Optimises active key lookup on each API request. |

## 6.4 Mongoose Schema (Reference)

```
const ApiKeySchema = new mongoose.Schema({
  name:      { type: String, required: true, unique: true },
  key:       { type: String, required: true, unique: true },
  isActive:   { type: Boolean, default: true },
  usageStats: [{
    date:     { type: Date, default: Date.now },
    endpoint: String,
    method:   String,
    ip:       String
  }]
}, { timestamps: true });
```

# 7. Collection Quick Reference

The table below summarises all collections, their primary keys, foreign key references, and notable indexes.

| Collection | PK | Foreign Keys | Unique Indexes | Other Indexes |
|---|---|---|---|---|
| users | _id | — | email | — |
| profiles | _id | user → users._id | user, email | — |
| bids | _id | user → users._id | user + targetDate | targetDate |
| apikeys | _id | — | name, key | isActive |